MedImages.jl: A Julia Package for Standardized Medical Image Handling and Spatial Metadata Management

Jakub Mitura[1*], Divyansh Goyal[1], Joanna Wybranska[1]

[1] JuliaHealth
[*] Corresponding author: jakub.mitura@gmail.com

Abstract

Quantitative medical imaging requires software that ensures metadata integrity and high-performance computing. MedImages.jl introduces a unified, differentiable ecosystem for 3D image analysis, providing native Julia implementations for spatial transformations while preserving clinical metadata through a BIDS-inspired architecture. By enforcing zero-cost metadata abstractions via Julia's type system, MedImages.jl eliminates the performance bottlenecks and metadata loss common in hybrid Python/C++ frameworks (SimpleITK, MONAI). The package achieves C++-level performance through LLVM JIT compilation while enabling end-to-end automatic differentiation—a capability unavailable in wrapped C++ libraries. Utilizing ITK only for robust I/O, the framework ensures full composability with scientific machine learning tools (Lux.jl, Zygote.jl) and supports efficient HDF5-based storage. Benchmarks demonstrate 30–138× GPU speedups (and up to 8.1× over SimpleITK CPU) while maintaining absolute clinical metadata integrity. MedImages.jl is open-source, supporting Linux, macOS, and Windows.

Author Summary

Medical imaging research often relies on Python for ease of use but requires C++ for performance, creating a barrier known as the "two-language problem." We present MedImages.jl, a Julia package that bridges this gap, providing high-performance, differentiable medical image processing with strong metadata support. This tool is particularly relevant for Nuclear Medicine, where large, functional images (SPECT, PET) and complex metadata are common. By enabling fast, parallelized computations on standard hospital hardware and supporting diverse data types (dose maps, CT), MedImages.jl democratizes advanced analysis for researchers and clinicians, ensuring that computational limitations do not hinder medical progress.

Introduction

Three converging challenges motivate the development of MedImages.jl:

Challenge 1: The Computational Scale of Modern Medical Imaging. Nuclear medicine and hybrid imaging generate among the largest and most metadata-intensive datasets in clinical practice. A single whole-body PET/CT examination produces two co-registered 3D volumes—one functional (PET), one anatomical (CT)—that must be spatially aligned, resampled to a common grid, and quantitatively normalized (e.g., Standardized Uptake Value conversion) before any downstream analysis can begin. Theranostic workflows further compound this complexity: a single treatment cycle may involve SPECT acquisitions with and without attenuation correction, voxel-wise absorbed dose maps, and diagnostic CT, each on its own voxel grid, orientation, and datatype. At biobank scale—hundreds to thousands of subjects—this preprocessing becomes the dominant bottleneck. In traditional hybrid Python/C++ environments (SimpleITK [**?**], MONAI [**?**]), loading and resampling a single whole-body PET/CT pair on a standard clinical workstation takes 3–5 seconds, and I/O overhead with metadata synchronization can account for over 60% of the total per-subject processing time. Cumulatively, this friction translates to hours of delay for studies involving the autoPET-III-Lite dataset [**?**] and similar multimodal repositories.

Challenge 2: The Two-Language Problem in Medical Image Computing. The Julia programming language [**?**] has established itself as a high-performance alternative for scientific computing, achieving C/Fortran-level speed through LLVM JIT compilation while maintaining the expressiveness of Python or MATLAB [**?**, **?**]. In adjacent fields, Julia has already produced impactful research tools: KomaMRI.jl for MRI pulse-sequence simulation [**?**], MRIReco.jl for MRI reconstruction [**?**], Lux.jl for deep learning [**?**], and DifferentialEquations.jl for ODE/PDE solving [**?**]. Critically, Julia's compiler-level automatic differentiation frameworks—Zygote.jl [**?**] and Enzyme.jl [**?**, **?**]—can differentiate through arbitrary code, including GPU kernels and numerical solvers, without hand-written adjoint implementations. This composability is especially valuable for emerging Scientific Machine Learning (SciML) paradigms, where physics-informed models, spatial transformations, and neural networks must coexist in a single differentiable pipeline. However, this potential remains unrealized in medical imaging because researchers working in Julia currently lack a foundational library for loading, transforming, and managing 3D medical volumes with clinical metadata integrity.

Challenge 3: The Missing Foundation—No Medical Imaging Library for Julia. While Julia's ecosystem excels in numerical computing, MRI simulation, and deep learning, there is currently no established package that provides the core infrastructure of medical image analysis: robust I/O for clinical formats (NIfTI, DICOM), spatial transformations with metadata preservation (resampling, rotation, registration), quantitative computations (SUV normalization, dose calculation), and GPU acceleration—all within a single, coherent framework. Existing Python-based solutions such as SimpleITK and MONAI solve parts of this problem but are fundamentally constrained by the Python-C++ bridge: researchers prototype in Python but rely on opaque, compiled C++ kernels for performance. This structural divide introduces what we term the "metadata preservation crisis"—when 3D volumes are converted to raw memory buffers (NumPy arrays) for neural network consumption, the critical spatial context (origin, spacing, direction

cosines) is decoupled from the voxel data, forcing manual synchronization and introducing "metadata drift" where transformations applied to the data are not reflected in the clinical parameters.

MedImages.jl addresses all three challenges simultaneously: it provides a unified, GPU-accelerated, differentiable medical imaging framework for Julia, where metadata is intrinsic to the data structure via zero-cost type abstractions, and every spatial operation—from resampling to affine registration—is natively composable with Julia's automatic differentiation and scientific computing ecosystem.

Architectural Foundations and the Metadata Preservation Crisis

At the core of medical imaging is the requirement that data must be defined within a physical coordinate system. Unlike standard computer vision, where an image is a simple array of pixels, a medical image is a grid of points occupying a specific region in space, defined by its origin, spacing, and direction cosines. The fundamental tenet of ITK and SimpleITK is the strict encapsulation of these parameters within a centralized image struct. In the C++ paradigm, these metadata are intrinsic to the object, ensuring that any filter applied to the voxel grid also correctly updates the spatial properties.

The crisis begins when these objects are integrated into Python-based deep learning workflows. While SimpleITK maintains metadata within its internal C++ `itk::Image` object, the common practice of converting these images to NumPy arrays for consumption by neural networks—using methods like `GetArrayFromImage`—strips the data of its spatial context. NumPy arrays are essentially raw memory buffers without native support for origin or orientation. This decoupling forces researchers to manually track metadata in parallel variables, a process prone to "metadata drift," where transformations applied to the voxel data during augmentation or preprocessing are not precisely reflected in the spatial parameters. This architectural friction is a key driver behind the development of MedImages.jl's typed approach.

Clinical Relevance and Multimodality Support

MedImages.jl is designed as a general-purpose library for 3D medical imaging, providing robust support for classical modalities such as Computed Tomography (CT) and Magnetic Resonance Imaging (MRI), as well as advanced molecular based techniques. While physics-informed deep learning is a key capability enabled by Julia's differentiability, the library is equally suited for standard image processing pipelines in nuclear medicine, radiology and oncology. Currently, it supports all major 3D imaging studies with the exception of ultrasonography.

Relevance to Nuclear Medicine

MedImages.jl is tailored for the complex requirements of Nuclear Medicine (NM), where radiotracers visualize physiological functions at cellular levels, often detecting disease before anatomical changes appear. NM scans, such as whole-body PET/SPECT, generate massive volumetric datasets due to

their extensive physical coverage. MedImages.jl addresses this by providing the high-performance computing and metadata integrity necessary for the functional and molecular characterization of disease processes.

Conventional imaging modalities primarily rely on morphological information—changes in size, shape, and tissue density [**?**, **?**]. While MRI offers superior soft-tissue contrast [**?**], these methods often struggle to identify small-volume disease or differentiate between benign and malignant tissue changes [**?**, **?**]. Nuclear medicine fills this gap by using radiotracers that target specific biological pathways, allowing for higher sensitivity and precision medicine (theranostics) [**?**, **?**].

The field encompasses diverse imaging types and modalities for the diagnostic of diverse diseases:

- **Positron Emission Tomography (PET):** The cornerstone of modern nuclear medicine, often combined with low dose or diagnostic CT or MRI. Different tracers interrogate different processes, such as glucose metabolism (18F-FDG) [**?**, **?**], cell membrane synthesis (Choline) [**?**, **?**], and antigen-specific targeting (PSMA) which has revolutionized prostate cancer management [**?**, **?**].

- **Single Photon Emission Computed Tomography (SPECT) and Theranostics:** A versatile workhorse for screening and staging, SPECT is increasingly used in theranostics to integrate diagnosis and therapy by pairing radiotracers (e.g., PSMA-targeted agents) that enable precision treatment of the same biological receptor [**?**, **?**, **?**].

- **Hybrid Imaging (PET/MRI):** Combines molecular sensitivity of PET with superior soft-tissue contrast of MRI—a morphological alternative to CT, reducing radiation exposure and potentially improving local staging [**?**, **?**].

Relevant Metadata in Nuclear Medicine

Quantitative nuclear medicine relies on the rigorous preservation of diverse metadata types, categorized into spatial, clinical, and temporal domains. Any loss or drift in these parameters can invalidate downstream diagnostic interpretation.

Standardized Uptake Value (SUV) and Clinical Parameters The SUV serves as a critical semi-quantitative metric for PET/CT, transforming visual assessments into objective data for diagnosis and therapeutic monitoring. The calculation is a physics problem dependent on specific clinical metadata:

$$SUV = \frac{C(t)}{ID/BW} \tag{1}$$

where $C(t)$ is the radioactive concentration at time t (Bq/mL), ID is the injected dose, and BW is the patient's body weight. $C(t)$ must be decay-corrected based on the tracer half-life and the time elapsed since injection:

$$C(t) = C_{measured} \cdot e^{\lambda \cdot \Delta t} \tag{2}$$

Losing parameters such as *Acquisition Time*, *Injected Dose*, or *Patient Weight* renders the image quantitatively useless. MedImages.jl enforces the preservation of these clinical fields within the typed image struct.

Spatial Integrity and Physical Range In whole-body surveys (e.g., PET/CT from mid-thigh to head), the alignment between functional and anatomical data relies on the physical integrity of the coordinate systems. Frameworks utilizing dictionary-based metadata management risk "metadata drift" if spatial keys are not updated in synchronization with voxel data during transformations. MedImages.jl's unified design prevents this drift by coupling the voxel grid directly to its spatial orientation metadata.

Temporal and Dynamic Metadata Beyond static volumes, dynamic PET (4D) and time-series data capture radiotracer kinetics, requiring precise management of frame start times, durations, and decay factors. MedImages.jl's `BatchedMedImage` facilitates these complex temporal associations, enabling differentiable kinetic modeling.

Clinical Applications of SUV Key domains where quantitative SUV calculations are essential include:

- **Disease Discrimination:** Distinguishing malignant lesions from physiological uptake [**?**, **?**].
- **Tumor Aggressiveness:** Correlating quantitative uptake with histopathological risk [**?**, **?**].
- **Staging and Prediction:** Utilizing primary tumor SUV to predict metastatic risk [**?**, **?**].
- **Theranostics:** Mandatory patient selection for targeted radioligand therapy [**?**].
- **Response Monitoring:** Objectively assessing response to systemic therapies [**?**, **?**].
- **Institutional Standardization:** Enabling cross-center data comparison [**?**].

MedImages.jl mitigates computational bottlenecks in clinical environments by enabling high-performance, parallelized processing that efficiently utilizes hospital hardware.

Design and Implementation

Software Architecture: Solving the Two-Language Barrier

MedImages.jl implements a BIDS-inspired metadata paradigm where the `MedImage` struct enforces metadata integrity through Julia's type system. Unlike Python workflows where converting to arrays discards spatial context, MedImages.jl uses zero-cost abstractions—metadata (origin, spacing, direction, patient ID, acquisition timestamp) is intrinsic to the structure without runtime overhead.

A critical design decision: ITK (C++) is used *only* for I/O robustness (NIfTI, DICOM parsing), while *all* transformations are native Julia. This ensures: (1) **Differentiability**—operations integrate with Zygote.jl/Enzyme.jl; (2) **Composability**—seamless integration with Lux.jl

and DifferentialEquations.jl; (3) **Transparency**—users can inspect and extend algorithms without C++ recompilation.

To support high-throughput analysis and address the "synchronization tax" of multimodal studies, MedImages.jl introduces `BatchedMedImage`, which stores 4D voxel data (x, y, z, batch) and vectorized metadata. This structure allows efficient batched processing where identical geometric operations (e.g., resampling PET to match CT orientation and spacing) are performed in a single GPU pass. To minimize the loading penalty for large datasets, the framework integrates HDF5-based storage, which outperforms sequential NIfTI I/O by enabling tiled, memory-resident groupings of subject tensors.

Functionalities and I/O Robustness

MedImages.jl provides a comprehensive set of functionalities optimized for 3D medical image processing:

- **Robust I/O:** Loading of various medical image formats (NIfTI, DICOM) using `ITKIOWrapper`.
- **Native Julia Transformations:** Functions to modify spatial properties of 3D volumes (`rotate_mi`, `translate_mi`, `scale_mi`, `resample_to_spacing`, `crop_mi`, `pad_mi`).
- **Resample to Target:** Resamples a moving image to match the spatial geometry of a reference image.
- **GPU Acceleration:** All transformations transparently support GPU execution via KernelAbstractions.jl.
- **HDF5 Support:** Fast loading and saving of 3D images using HDF5, speeding up I/O for intermediate processing and reducing the "loading penalty" common in deep learning pipelines.
- **Autodifferentiation:** All spatial transformations have well-defined gradients via Zygote.jl and Enzyme.jl.
- **Orientation Management:** Robust tools to handle and convert between neuroimaging coordinate systems.

Results

Performance Benchmarks

MedImages.jl was benchmarked on CPU and GPU backends. Benchmarks were conducted on an NVIDIA RTX 3090 GPU and Intel Core Ultra 9 285K CPU.

For $256 \times 256 \times 128$ volumes, MedImages.jl on GPU achieves significant speedups compared to SimpleITK on CPU:

- **Resampling (Nearest Neighbor):** 115× speedup vs CPU, 17× vs SimpleITK.
- **Orientation Changes:** 71× speedup vs CPU, 31× vs SimpleITK.
- **Fused Affine Transformation:** 135× speedup vs CPU, 8.1× vs SimpleITK.

These results demonstrate that GPU acceleration is essential for 3D medical imaging, particularly for the large, multimodal datasets common in Nuclear Medicine.

Scaling to 100 Cases: MedImages (HDF5) vs. MONAI

To evaluate the framework under high-throughout production loads, we benchmarked a 100-case preprocessing pipeline involving PET/CT alignment and resampling to a $512 \times 512 \times 512$ grid. For a fair comparison with MONAI's `PersistentDataset`, we evaluate performance for high-frequency training loops (Epochs 2+), assuming data has been cached or pre-loaded.

Table 1: Definitive Performance Comparison (100 Subjects, Epochs 2+)

Phase (per Subject)	**MedImages (HDF5)**	**MONAI (PersistentDataset)**
Stage 1: Load from Cache	**40 ms**	150 ms
Stage 2: Transform (Compute)	**50 ms**	500 ms
Total Turnaround	**90 ms**	**650 ms**

Overall, MedImages.jl achieved a 7.2× total turnaround speedup compared to the MONAI PersistentDataset baseline in this high-throughput scenario.

SUV Precision and Metadata Integrity

We verified the quantitative accuracy of MedImages.jl by comparing its Standardized Uptake Value (SUV) calculations against the gold standard SimpleITK. Due to Julia's high-precision handling of temporal and clinical metadata, MedImages.jl achieves identical results for SUVbw (Body Weight) calculations, matching SimpleITK's output up to 10^{-14} in double precision. This ensures that the massive speedups achieved through GPU acceleration do not compromise clinical validity.

Furthermore, we conducted an end-to-end consistency experiment on the `autoPET-III-Lite` dataset to verify that metadata preservation and quantitative fidelity are maintained throughout complex spatial transformation pipelines. The evaluation involved a multi-modal sequence consisting of (1) converting PET intensities to SUV, (2) resampling the PET and label masks to the native CT target space, (3) changing the spacing to a 2.0 mm isotropic grid, (4) applying a 45-degree rotation around the Z-axis, and (5) applying a 10-voxel translation along the X-axis. For the segmentation masks, nearest-neighbor interpolation was strictly applied stringently to prevent boundary blurring, while linear interpolation was utilized for the continuous PET/CT intensities.

Quantitative analysis of the Mean SUV and total Lesion Volume inside the mask demonstrated remarkable consistency before and after these compound transformations. The original Mean SUV was 3.4913 with a volume of 52.55 cm^3, and the final transformed volume exhibited a Mean SUV of 3.4660 and a volume of 51.86 cm^3. The minor deviations in Mean SUV (0.73%) and Volume (1.32%) are fundamentally attributable to grid resampling artifacts and interpolation rather than underlying metadata deterioration. Figure 1 visually illustrates the multi-modal alignment before and after the full transformation pipeline.

figures/suv_consistency.png

Figure 1: **Quantitative Consistency Under Compounded Spatial Transformations (Sagittal Views).** The four-panel composite shows sagittal slices of the same subject before and after a sequence of spatial transformations (isotropic 2.0 mm resampling, 45° Z-rotation, X-translation): **Top-left**: CT after transformations; **Top-right**: PET/SUV after transformations; **Bottom-left**: CT in original native space; **Bottom-right**: PET/SUV in original native space. MedImages.jl preserves spatial alignment and clinical metadata throughout, with Mean SUV and gross lesion volume varying by $< 1.5\%$ across the full pipeline.

Automatic Differentiation and Scientific Machine Learning

Scientific Machine Learning (SciML) is a paradigm that embeds mechanistic, physics-based models—such as ordinary and partial differential equations, conservation laws, and domain-specific constraints—directly into the training loop of neural networks, rather than treating the model as a purely data-driven black box. The Julia ecosystem is uniquely positioned for SciML because its compiler-level automatic differentiation (AD) frameworks (Zygote.jl [**?**], Enzyme.jl) can differentiate through arbitrary Julia code, including numerical ODE/PDE solvers (DifferentialEquations.jl), GPU kernels (KernelAbstractions.jl), and custom spatial transformations, without requiring hand-written C++/CUDA adjoint code. This composability means that a single Julia program can express an end-to-end pipeline—from

image loading, through physics-informed spatial operations, to a neural network loss function—while maintaining a fully differentiable computational graph.

MedImages.jl leverages this capability directly: all spatial transformations (rotation, resampling, affine warping) are implemented as native Julia kernels compatible with AD. This allows gradients to flow from a downstream loss function back through the geometric operations and into the imaging physics, enabling applications such as differentiable registration, learned augmentation policies, and neural network–driven reconstruction.

Proof-of-Concept: Learned Inverse Rotation via Spatial Transformer. To validate end-to-end differentiability, we implemented a neural-network-based spatial transformer that learns to predict and invert unknown 3D rotations applied to volumetric data. The pipeline operates as follows:

1. **Synthetic Data Generation:** A primitive volumetric pattern (a central bar along the Z-axis in a 32^3 grid) is rotated by random Euler angles $(\alpha_x, \alpha_y, \alpha_z) \in [-45°, 45°]$ using MedImages.jl's `affine_transform_mi` to produce the input batch X. The target batch Y is the original, unrotated volume.

2. **Architecture:** A Lux.jl 3D CNN consisting of two convolutional layers (`Conv(`5^3`, 1→8)` → `MaxPool(`2^3`)` → `Conv(`3^3`, 8→16)` → `MaxPool(`2^3`)`) followed by a fully-connected head (`Dense(8192→64)` → `Dense(64→3)`) predicts three Euler angles per batch item.

3. **Differentiable Geometric Pipeline:** The predicted angles are converted to Rodrigues rotation matrices via `Rodrigues_rotation_matrix`, composed into a 4×4 affine matrix, and used to generate a differentiable sampling grid. MedImages.jl's native `interpolate_pure` function then resamples the rotated input at the computed grid coordinates using trilinear interpolation.

4. **Training:** The MSE loss between the de-rotated output and the original target Y is minimized using Zygote.jl backpropagation and the Adam optimizer. Crucially, gradients flow from the loss *through* the trilinear interpolation kernel and the matrix construction back into the CNN weights—a capability unavailable in frameworks that rely on non-differentiable C++ resampling backends.

Over 20 training iterations, the spatial transformer consistently reduced reconstruction error by over 65

Differentiability Verification

To rigorously validate that automatic differentiation produces correct gradients through every spatial operation, we implemented a comprehensive test suite covering all core transformations. For each operation—`resample_to_image`, `rotate_mi`, `scale_mi`, `translate_mi`, `crop_mi`, `pad_mi`, `resample_to_spacing`, and `change_orientation`—we defined a scalar loss function $\mathcal{L}(\mathbf{x}) = \sum f(\mathbf{x})$, where f is the spatial transformation applied to input voxel data $\mathbf{x}$, and verified that Zygote.jl returns non-trivial, structurally correct gradients.

Specifically, for identity-preserving operations such as `translate_mi` (integer-voxel shifts) and `pad_mi` (zero-padding), we verified that $\nabla_{\mathbf{x}}\mathcal{L} = \mathbf{1}$, confirming that the gradient propagates without corruption. For spatially

selective operations like `crop_mi`, we verified that the gradient is 1 inside the cropped region and 0 outside. For interpolation-based transforms (`rotate_mi`, `resample_to_image`, `resample_to_spacing`), we verified gradient existence and non-triviality. Batched operations were separately tested to confirm that gradients flow correctly through the 4D data path, including through the `create_batched_medimage` constructor. Additionally, for the fused affine interpolation kernel, we validated CPU/GPU gradient consistency to within 10^{-4}, ensuring that the CUDA kernel produces numerically equivalent gradients to the CPU reference implementation. This systematic verification across 7 test files establishes a foundation of trust for deploying MedImages.jl in gradient-based optimization pipelines.

Organ-Specific Affine Registration with Fused GPU Loss

To demonstrate MedImages.jl's capability for production-level scientific machine learning, we developed a deep learning pipeline for multi-organ affine registration. Given a CT scan and an atlas segmentation mask, the model predicts per-organ affine transformation parameters and is trained end-to-end using a custom differentiable GPU loss kernel.

Model Architecture. The network, termed `MultiScaleCNN`, is a 3D CNN built with Lux.jl [**?**]. It consists of three parallel convolutional branches with kernel sizes 3^3, 5^3, and 7^3, each containing two strided convolutions (stride 2) followed by global mean pooling. The concatenated 96-dimensional feature vector is processed by a two-layer MLP ($96 \to 256 \to 256 \to N_{\text{organs}} \times 15$). The output is reshaped to $(15, N_{\text{organs}}, B)$ and constrained: rotation parameters are bounded to $[-\pi, \pi]$ via tanh, scale factors are enforced positive via exp, shear coefficients are bounded to $[-1, 1]$ via tanh, while translation and center-of-rotation parameters remain unconstrained.

Fused Loss Kernel with Tree Reduction. The loss computation is implemented as a single KernelAbstractions.jl GPU kernel that fuses three operations into one pass over 512 atlas points per organ:

1. **Affine Transform Application:** Each atlas point is transformed by the predicted affine parameters (scaling $\to$ shearing $\to$ sequential $R_x R_y R_z$ rotation $\to$ translation) to yield transformed coordinates in the target volume space.

2. **Distance Penalty:** A $\text{Softplus}(\|p - \bar{c}\| - r_{\max})$ penalty constrains transformed points to remain within the organ's anatomical region, where $\bar{c}$ is the gold-standard barycenter and $r_{\max}$ is the maximum radius.

3. **Interpolation Loss:** Trilinear interpolation samples the one-hot-encoded gold standard volume at the transformed coordinates, and a squared error $(1 - v_{\text{interp}})^2$ penalizes misalignment.

Intra-workgroup reduction uses binary tree reduction in shared memory (512 threads per workgroup), avoiding atomic operations entirely. One workgroup processes one (organ, batch) pair, with the total loss averaged across all organs and batch items.

Enzyme-Based Gradient Computation. A custom `ChainRulesCore.rrule` wraps the kernel launch and calls `Enzyme.autodiff(Reverse, ...)` to differentiate the entire GPU kernel. This allows Zygote.jl to treat the fused loss as a standard differentiable function: gradients flow from the scalar loss through the GPU kernel back to the predicted affine parameters, and from there through the Lux.jl CNN to update all network weights via the Adam optimizer. The pipeline supports multi-GPU training via MPI. This experiment validates that MedImages.jl's kernel-level differentiability enables novel GPU-accelerated registration architectures that would be extremely difficult to implement in Python/C++ frameworks requiring hand-written CUDA adjoint code.

Fused Affine Preprocessing Pipeline

For large-scale studies requiring standardized preprocessing of heterogeneous medical volumes, we developed a fused affine preprocessing pipeline that collapses multiple spatial transformations—orientation normalization, spacing resampling, and spatial centering—into a single 4×4 affine matrix. This matrix is precomputed on the CPU from the source and target spatial metadata, then applied via a custom KernelAbstractions.jl kernel with unrolled trilinear interpolation on the GPU.

The kernel maps each output voxel index to a source coordinate via the fused matrix, performs bounds checking, and executes either nearest-neighbor or trilinear interpolation. A custom `ChainRulesCore.rrule` with `Enzyme.autodiff` enables gradient flow through the entire preprocessing step, making it suitable for integration into differentiable training loops where the preprocessing itself must be part of the computational graph. Benchmarking on 100 subjects resampled to a $512 \times 512 \times 512$ target grid demonstrated per-subject GPU kernel times of under 200 ms, confirming the pipeline's suitability for biobank-scale processing.

Illustrative Example: Multi-Modal Batched Processing in Theranostics

Theranostic radioligand therapy workflows integrate longitudinal patient imaging acquired at different time points, on different devices and reconstruction pipelines—e.g., [^{177}Lu]Lu-PSMA SPECT with and without attenuation correction (AC/NAC), voxel-wise absorbed dose maps, and CT—where each modality retains its own voxel grid, datatype, and spatial metadata (spacing, origin, orientation).

To demonstrate the capability of `BatchedMedImage` in handling complex, multi-modal datasets typical for theranostics workflows, we processed a dataset of four typical examinations, each containing a distinct imaging modality:

^{177}Lu Lu-PSMA SPECT with attenuation correction (AC)

^{177}Lu Lu-PSMA SPECT without attenuation correction (NAC)

^{177}Lu Lu-PSMA Dosemap (voxel-wise absorbed dose)

1. Computed Tomography (CT) for anatomical reference

The images were loaded into a single `BatchedMedImage` object, preserving their individual spatial metadata and voxel types. A batched rotation

transformation was then applied simultaneously to all four modalities. Figure 4 illustrates the results, showcasing the original axial slices (left) and the rotated output (right).

The ability to apply identical geometric transformations across functional (SPECT, Dosemap) and anatomical (CT) data in a single operation ensures perfect spatial alignment is maintained during augmentation or registration tasks, which is critical for accurate dosimetry and treatment planning.

`figures/orig.png`

`figures/rot.png`

Figure 2: *
A: Original

Figure 3: *
B: Rotated

Figure 4: **Simultaneous Batched Rotation of Multi-Modal Theranostic Data.** Panel A displays the original axial slices of four distinct modalities stored within a single `BatchedMedImage` structure: (Top-Left) [^{177}Lu]Lu-PSMA SPECT AC, (Top-Right) [^{177}Lu]Lu-PSMA SPECT NAC, (Bottom-Left) [^{177}Lu]Lu-PSMA Dosemap, and (Bottom-Right) CT. Panel B shows the result after applying a single `rotate_mi` operation to the batch. The spatial integrity and alignment between the functional distribution (SPECT/Dose) and anatomical structure (CT) are preserved during the transformation.

1 Discussion

As the field of nuclear medicine moves toward more complex 4D dynamic studies and scientific machine learning, the architectural limitations of traditional Python/C++ frameworks will become increasingly apparent. Our results demonstrate that the MedImages.jl ecosystem provides a tangible path forward by solving both the performance bottlenecks and the metadata preservation crisis.

Comparative Metadata Architectures: Beyond the Python-C++ Divide

Modern Python frameworks like MONAI [**?**] and TorchIO [**?**] have introduced sophisticated mechanisms to combat metadata decoupling, yet they remain tethered to the underlying limitations of the Python-C++ bridge. MONAI utilizes a MetaTensor structure, which subclasses the PyTorch Tensor and appends a metadata dictionary. While this allows metadata

to travel with the tensor, the implementation relies on Python dictionaries, which introduce significant runtime overhead compared to compiled structs (as seen in our 7.2× speedup benchmarks). Furthermore, MONAI's metadata management is often a reactive layer; many of its core geometric transformations default to backends like SciPy or SimpleITK, meaning the actual computation occurs in a non-differentiable "black box," breaking the autograd graph.

TorchIO approaches the problem through a hierarchical Subject class system. While effective for multi-modal synchronization, TorchIO is primarily a manager of third-party backends (SimpleITK, NiBabel). This dependency chain maintains a layer of indirection that can lead to memory management issues in large-scale training.

In contrast, MedImages.jl represents a paradigm shift by leveraging Julia's type system to implement a BIDS-inspired metadata paradigm. As shown in Table 2, metadata is an intrinsic part of a strictly typed `MedImage` struct. Because Julia compiles to LLVM machine code, these metadata abstractions are "zero-cost"; the compiler can optimize access to parameters like origin and spacing exactly as it would for raw integers, eliminating the runtime overhead associated with Python's dictionary lookups or C++ object-oriented indirection.

This zero-cost abstraction directly translates to the performance gains observed in our benchmarks. The 7.2× advantage over MONAI's `PersistentDataset` is primarily attributed to MedImages.jl's structured HDF5 tile-reading—which bypasses the metadata overhead and serialization costs inherent in Python's pickle-based caching—and its single-pass Fused Matrix-Affinity kernel, which collapses multiple spatial transformations into a single CUDA execution.

Table 2: Comparative Metadata Architectures in Modern Frameworks

Framework	**Metadata Storage Mechanism**	**Preservation Strategy**	**Known Interoperability Gaps**
SimpleITK	Internal C++ Struct	Encapsulated in `itk::Image`	Discarded upon conversion to NumPy.
MONAI	MetaTensor (Dict-based)	Metadata dictionary moves with tensor	Dict lookup overhead; interop with ITK can flip orientations.
TorchIO	Subject Class Attributes	Multi-modal synchronization	Memory buildup during augmentation; backend dependency.
MedImages.jl	Typed Julia Struct	Zero-cost type abstractions	N/A (Native Julia stack throughout).

The Primacy of Clinical and Temporal Metadata

Quantitative nuclear medicine critically relies on intact clinical and temporal metadata: SUV computation depends explicitly on injection and acquisition times as well as patient-specific factors. In theranostic workflows, this requirement collides with a common preprocessing need—applying identical geometric operations across multiple co-referenced modalities (e.g., [^{177}Lu]Lu-PSMA SPECT AC/NAC, absorbed-dose maps, and CT) while preserving each modality's native voxel grid and spatial context. Conventional array-based pipelines enable "metadata drift," which can break

cross-timepoint correspondence and bias quantitative endpoints.

MedImages.jl addresses this by design: the `BatchedMedImage` abstraction allows single-call, batched geometric transforms to be applied consistently across modalities without manual bookkeeping. Our cross-platform validation confirms that the SUV conversion factors extracted by MedImages.jl achieve mathematical identity with established SimpleITK benchmarks to a precision of 10^{-14}. This numerical equivalence ensures that researchers transitioning to Julia can maintain absolute continuity with legacy clinical findings while gaining the performance required for massive biobank-scale analysis.

Practical Guidance: When to Use Which Framework

We do not advocate replacing established tools wholesale. Rather, we recommend choosing the framework that best matches the task at hand.

When MONAI and SimpleITK remain the pragmatic choice. For teams with substantial legacy Python codebases, switching frameworks carries non-trivial migration costs. MONAI provides a mature library of ready-made, clinically validated components—pretrained segmentation networks (e.g., SwinUNETR, SegResNet), standardized data-loading pipelines, and extensive augmentation transforms—that can be deployed with minimal custom code. Similarly, SimpleITK offers a comprehensive catalogue of classical filters (Gaussian smoothing, morphological operations, connected-component labelling) whose correctness has been validated across decades of clinical deployment. When the research question can be answered with an existing, well-tested pipeline and the primary requirement is rapid prototyping within the PyTorch ecosystem, these tools remain the most efficient choice.

When MedImages.jl offers a clear advantage. The Julia-based approach becomes compelling in several distinct scenarios:

- **Novel algorithm development.** When researchers need to implement a new spatial transformation, loss function, or physics-informed model from scratch, Julia's single-language stack eliminates the need to write C++/CUDA extensions. The algorithm can be prototyped, debugged, and GPU-accelerated within the same language, with full automatic differentiation support.

- **Maximum throughput at scale.** For biobank-level studies processing thousands of whole-body PET/CT volumes, the 7.2× turnaround advantage demonstrated here translates directly into reduced wall-clock time and hardware cost. The fused-kernel approach and HDF5-based I/O bypass the serialization overhead that becomes a bottleneck at scale.

- **Differentiable imaging physics.** Tasks such as learned registration, differentiable augmentation, or physics-informed neural networks require gradients to flow through spatial transformations. MedImages.jl's native Julia kernels integrate directly with Zygote.jl and Enzyme.jl, whereas MONAI and TorchIO must fall back on non-differentiable C++ backends for many geometric operations.

- **Cross-domain scientific computing.** Julia's ecosystem uniquely bridges medical imaging with adjacent scientific domains—DifferentialEquations.jl for pharmacokinetic modelling, KomaMRI.jl [**?**] for MRI pulse-sequence simulation, Flux.jl/Lux.jl [**?**] for deep learning—all within a single runtime. This composability enables workflows that would otherwise require fragile inter-process communication between Python, MATLAB, and C++ components.

- **Exploring unconventional approaches.** Researchers who wish to rethink standard medical imaging pipelines—for example, by embedding physical constraints directly into neural architectures or by coupling image reconstruction with downstream inference—benefit from Julia's transparency: every kernel is inspectable and modifiable Julia code, not an opaque compiled binary.

In practice, we envision a complementary workflow: established MONAI/SimpleITK pipelines for routine clinical deployment, and MedImages.jl for research frontiers where performance, differentiability, or algorithmic novelty are the primary requirements.

MedImages.jl offers a path forward—a future where imaging physics is integrated directly into the deep learning pipeline, where hardware acceleration is transparent and ubiquitous, and where the metadata that defines our physical world is preserved with the same rigor as the pixel values themselves. By bridging the divide between the physics of acquisition and the logic of inference, and by providing a 7.2× faster turnaround than traditional caching frameworks in large-scale multimodal studies, MedImages.jl empowers researchers and clinicians to unlock the full potential of medical imaging as a precise, quantitative science.

Conclusions

The development of MedImages.jl represents a fundamental shift in how we approach medical image analysis. By overcoming the "two-language problem" through Julia's unique capabilities, the library provides a unified framework where physical accuracy and computational performance are no longer at odds. The ability to perform differentiable spatial transformations while maintaining absolute metadata integrity is not just a technical achievement; it is a clinical necessity for the era of molecular imaging and quantitative theranostics.

Availability and Future Directions

MedImages.jl is open-source and available at `https://github.com/JuliaHealth/MedImages.jl`. It is licensed under the Apache License 2.0.

Future work will focus on:

1. Integrating with advanced reconstruction frameworks (KomaMRI.jl [**?**], GIRFReco.jl [**?**]) for end-to-end physics-informed learning.

2. Implementing differentiable registration algorithms leveraging Zygote.jl.

3. Expanding support for standardized formats like DICOM-SEG via Highdicom [**?**] integration.